F01: Importing Data

Related Use Case: UC01: Import Transactions

Related Sequence Diagram: SD1: Import transactions

Design Pattern(s): Factory, Singleton

Classes involved:

- GUI: This class is responsible for showing the import dialog and taking the user's choice of bank, account type, and file, and displays the preview and the summary.
- AgentController: receives the request from the GUI and passes it to the right service.
- ImportService: runs the import: gets a parser, validates rows, finds duplicates, and saves.
- ParserFactory: creates the parser that matches the chosen bank's CSV layout.
- StatementParser/CsvParser/TDParser/CIBCParser: the parser interface turns rows into transactions.
- ImportResult: holds the preview and the final clean summary
- Transaction: one parsed row to contain date, amount, description, and its account details.
- DatabaseManager: the one shared object that saves the transactions.

Important methods:

- AgentController.previewImport()
- ImportService.previewImport()
- ParserFactory.createParser()
- StatementParser.parse()
- ImportService.validateRow()
- ImportService.findDuplicates()
- AgentController.confirmImport()
- ImportService.confirmImport()
- DatabaseManager.save()

Execution: When the user picks a bank, an account type, and a CSV file, the GUI sends them to AgentController.previewImport(), which calls ImportService.previewImport(). The service asks ParserFactory.createParser(bank) for the parser that fits that bank. If the file is not a CSV or is empty, an error goes back to the user and nothing is imported. Otherwise StatementParser.parse() turns the rows into Transaction objects, validateRow() checks each row, and findDuplicates() finds repeats. The result comes back as an ImportResult and the GUI shows it as a preview. Nothing is saved yet. When the user clicks Confirm, confirmImport() saves the transactions with DatabaseManager.save(), and returns the summary.

F02: Automatic Categorization

Related Use Case: UC02: Categorize Transactions

Related Sequence Diagram: SD2: Categorize transactions

Design Pattern(s): Adapter, Singleton

Classes involved:

- AgentController: starts categorization after an import and serves the category summaries to the GUI.
- CategorizationService: applies the stored rules first and then proceeds to ask Claude about the remaining transactions
- CategoryRule: a saved pattern that maps a merchant to a category.
- Transaction: the transaction being labeled and the category it receives
- LLMClient / ClaudeClient: the interface the service talks to and the adapter that sends the request to Claude and reads the reply.
- DatabaseManager: saves the categories and loads transactions for the summaries.
- GUI: shows the categorized transactions with totals and percentages for the user to review.

Important methods:

- CategorizationService.categorize()
- CategorizationService.applyRules()
- CategorizationService.askClaude()
- LLMClient.generate()
- DatabaseManager.save()
- AgentController.getCategorySummaries()
- CategorizationService.getCategorySummaries()
- DatabaseManager.find()

Execution:

After an import, the controller calls CategorizationService.categorize(transactions). For each transaction, applyRules() tries the stored rules first. If some transactions are still unmatched, askClaude() sends their descriptions to Claude through LLMClient.generate(prompt), and the reply (a category) is parsed. If the agent times out, those transactions stay "Uncategorized". The results are saved with DatabaseManager.save(). When the user opens the transaction list, the GUI calls AgentController.getCategorySummaries(month), which loads the transactions through CategorizationService and returns each category's total and percentage of spending for review.

F03: Manual Category Correction

Related Use Case: UC03: Correct Transactions

Related Sequence Diagram: SD3: Correct a category

Design Pattern: Singleton

Classes involved:

- GUI: lets the user select a row, pick a category, and tick "Apply to similar transactions".
- AgentController: forwards the correction and triggers the budget recheck.
- CategorizationService: updates the transaction and, if asked, saves a rule for similar transactions.
- CategoryRule: the rule created from the transaction's description pattern.
- Transaction / Category: the transaction being changed and its new category.
- BudgetService: rechecks budget totals because a category total just changed.
- DatabaseManager: saves the updated transaction and the new rule.

Important methods:

- AgentController.recategorize()
- CategorizationService.recategorize()
- DatabaseManager.save()
- CategorizationService.saveRule()
- BudgetService.checkThresholds()

Execution:

The user selects a row, chooses "Edit Category", and picks a new category from the allowed list. If the user cancels, nothing is saved. Otherwise the GUI calls AgentController.recategorize(txnId, category, applyToSimilar), which calls CategorizationService.recategorize(). The service saves the updated Transaction through DatabaseManager.save(). If "Apply to similar" was ticked, saveRule(pattern, category, replace) creates a CategoryRule from the description pattern. If it conflicts with an existing rule, the user is asked whether to replace it or keep both. Otherwise the rule and the matching transactions are saved. Finally, the controller calls BudgetService.checkThresholds() and the GUI refreshes.

F04: Budget Creation

Related Use Case: UC04: Create a Budget

Related Sequence Diagram: SD4: Create a budget and receive alerts

Design Pattern: Singleton

Classes involved:

- GUI: collects the month, category, and amount, and shows the budget on the dashboard.
- AgentController: passes the request to the budget service.
- BudgetService: validates the request and saves the budget.
- Budget / Category: the monthly limit and the category it belongs to.
- DatabaseManager: saves the budget.

Important methods:

- AgentController.setBudget()
- BudgetService.setBudget()
- BudgetService.budgetExists()
- DatabaseManager.save()

Execution:

The user enters a month, a category, and an amount in the budget panel. The GUI calls AgentController.setBudget(month, category, amount, overwrite), which calls BudgetService.setBudget(). The service first calls budgetExists(). If the amount is zero or negative, it rejects the request and the user is prompted again. If a budget already exists and overwrite is false, the user is asked to overwrite or cancel. Otherwise the Budget is saved with DatabaseManager.save(), and the dashboard refreshes to show it.

F05: Budget Alerts

UC05: View Budget Alerts

SD4: Create a budget and receive alerts

Design Pattern: Observer, Singleton

Classes involved:

- AgentController: calls BudgetService.checkThresholds() after an import, a category correction, or a new budget.
- BudgetService: checks spending against budgets and notifies its observers.
- AlertObserver: the interface every alert receiver implements.
- GuiAlertPanel / CliAlertPrinter: the two observers: one updates the GUI's notification icon, the other prints to the terminal.
- GUI / CLI: own their observers and register them with the service.
- DatabaseManager: loads the month's transactions.

Important methods:

- BudgetService.attach()
- BudgetService.checkThresholds()
- BudgetService.getStatus()
- BudgetService.notifyObservers()
- AlertObserver.onAlert()
- BudgetService.detach()

Execution:

At start-up the GUI registers its GuiAlertPanel with BudgetService.attach(), and the CLI does the same with its CliAlertPrinter. When a transaction is added or changed, BudgetService.checkThresholds(month) loads the month's transactions and calls getStatus() for each category. If a category has no budget, it shows "no budget set". If a threshold is crossed for the first time that month, notifyObservers(message) loops over every attached observer and calls onAlert(message) on each, so the GUI and the CLI both hear about it without BudgetService knowing either exists. At shutdown, the GUI calls detach().

F06: Manage Subscriptions

Related Use Case: Manage Subscriptions

SD5: Manage subscriptions

Design Patterns: Adapter, Singleton

Classes involved:

- GUI: shows the subscription list and overlap message. Then, take the user's confirmation, rename, or mark choices.
- AgentController: serves the subscription list and overlap message to the GUI.
- SubscriptionServices: detects recurring charges, labels service types, finds overlaps, and saves the user's choices.
- Subscription / Transaction: the detected subscription and the repeating charges it was built from.
- LLMClient / ClaudeClient: ask Claude for a service type and for the overlap message.
- DatabaseManager: loads transactions and saves subscriptions.

Important methods:

- AgentController.getSubscriptions()
- SubscriptionService.detectSubscriptions()
- SubscriptionService.normalizeMerchant()
- SubscriptionService.classifyServiceType()
- LLMClient.generate()
- AgentController.getOverlapMessage()
- SubscriptionService.findOverlaps()
- AgentController.confirmSubscription()
- SubscriptionService.confirm()

Execution: When the user opens the Subscriptions tab, the GUI calls AgentController.getSubscriptions(), which calls SubscriptionService.detectSubscriptions(). The service loads the transactions. With no history it asks the user to upload a CSV, and with only one month it says more history is needed. Otherwise normalizeMerchant() cleans the names, charges with similar amounts at regular intervals are grouped into subscriptions, and classifyServiceType() labels each one, asking Claude through LLMClient.generate() only when the lookup table has no answer. The subscriptions are saved. The GUI then calls getOverlapMessage(), and findOverlaps() asks Claude to write the overlap message, which is left out if the agent is unavailable. When the user confirms a subscription, AgentController.confirmSubscription() calls SubscriptionService.confirm(), which saves the choice.

F07: Natural Chatbox with Agent

UC07: Natural Chatbox

SD6: Chat with the agent

Design Pattern: Adapter, Singleton

Classes involved:

- GUI: the chat panel where the user types a question and reads the answer.
- AgentController: sends the question to the agent.
- FinanceAgent: runs the loop which involves asking Claude, running the tool Claude picks, and repeats until it has an answer.
- SpendingTool: the tool interface and the tools that fetch real numbers from the user's data.
- LLMClient / ClaudeClient / AnthropicApi: the interface, its adapter, and Anthropic's API.
- DatabaseManager: where the tools read the data from.

Important methods:

- AgentController.askQuestion()
- FinanceAgent.run()
- LLMClient.generate()
- FinanceAgent.executeTool()
- Tool.execute()
- DatabaseManager.find()
- FinanceAgent.stepLimitReached()

Execution:

The user types a question, and the GUI calls AgentController.askQuestion(text), which calls FinanceAgent.run(question). The agent adds the question to its memory. If the agent is unavailable, a timeout shows "Agent is down". Otherwise it loops: LLMClient.generate(prompt) asks Claude what to do, and Claude replies with either a tool call or the final answer. For a tool call, executeTool(name, args)

runs the matching Tool.execute(args), for example SpendingTool, which reads the data with DatabaseManager.find() and returns the result for the next step. The loop ends when Claude writes the answer or stepLimitReached() is true. The answer is added to memory and returned through the controller to the GUI.

F08: Savings Goal Management

UC08: Saving Goals

SD8: Manage a savings goal

Design Pattern: State, Adapter, Singleton

Classes involved:

- GUI: collects the goal details and contributions and shows progress and messages.
- AgentController: forwards goal requests to the goal service.
- GoalService: creates goals, records contributions, and reports status.
- Goal: holds the goal's data and forwards its actions to its current state object.
- GoalState / InProgressState / CompletedState: the state interface and its two implementations. Only InProgressState asks Claude for tips.
- LLMClient / ClaudeClient: let the in-progress state ask Claude for catch-up tips.
- DatabaseManager: saves and loads goals.

Important methods:

- AgentController.createGoal()
- GoalService.createGoal()
- AgentController.addContribution()
- GoalService.addContribution()
- Goal.addContribution()
- InProgressState.addContribution()
- Goal.updateState()
- Goal.setState()
- AgentController.getGoalStatus()
- GoalService.getStatus()
- Goal.getMessage()
- InProgressState.getMessage()
- CompletedState.getMessage()
- LLMClient.generate()

Execution:

Creating a goal. The user enters a name, a target, and a deadline. The GUI calls AgentController.createGoal(), which calls GoalService.createGoal(). If the target is zero or less, the service rejects it. Otherwise it saves the goal.

Adding a contribution. The GUI calls AgentController.addContribution(goalId, amount), which calls GoalService.addContribution(). The service loads the Goal and calls Goal.addContribution(amount). The goal passes this to its current state object, InProgressState.addContribution(). The goal then calls updateState() to check whether the target has been reached. If it has, setState(CompletedState) switches the goal to the completed state. Finally the service saves the goal.

Checking the status. The GUI calls AgentController.getGoalStatus(), which calls GoalService.getStatus(), which calls Goal.getMessage(). The goal passes this to its current state. If the goal is in progress, InProgressState.getMessage() asks Claude for catch-up tips through LLMClient.generate(), but only when the goal is behind. If the goal is completed, CompletedState.getMessage() returns "goal reached" and the required contribution is no longer shown.

F09: Monthly Report Generation

UC09: Generate Monthly Report

SD10: Generate a monthly report

Design Pattern: Adapter, Singleton

Classes involved:

- GUI: collects the month and displays the finished report.
- AgentController: forwards the request to the report service.
- ReportService: gathers the facts, asks Claude for the summary, and saves the report.
- BudgetService / SubscriptionService: supply the budget status and the subscription changes.
- MonthlyReport: totals, category breakdown, and summary.
- LLMClient / ClaudeClient: ask Claude to write the summary.
- DatabaseManager: loads the month's transactions and saves the report.

Important methods:

- AgentController.generateMonthlyReport()
- ReportService.generate()
- DatabaseManager.find()
- BudgetService.getStatus()
- ReportService.findUnusualTransactions()
- SubscriptionService.detectSubscriptions()
- ReportService.writeSummary()
- LLMClient.generate()
- ReportService.templateSummary()
- DatabaseManager.save()

Execution:

The user selects a month and clicks Generate. The GUI calls AgentController.generateMonthlyReport(month), which calls ReportService.generate(month). The service loads the month's transactions with DatabaseManager.find(). If there are none, it returns a warning. Otherwise it collects the facts with code: BudgetService.getStatus() for budget performance, findUnusualTransactions() for unusual charges, and SubscriptionService.detectSubscriptions() for subscription changes. Then writeSummary(facts) asks Claude through LLMClient.generate() to write the summary, and if the agent is unavailable templateSummary() produces a plain one instead. The MonthlyReport is saved with DatabaseManager.save() and shown on screen.

F10: Report Export

Related Use Case: UC10: Export Report

Related Sequence Diagram: SD9: Export a report

Design Pattern(s): None

Classes involved:

- GUI: collects the report type, month, format, and save location, and shows where the file was written.
- AgentController: forwards the export request.
- ReportService: checks the path and writes the file with the helper that matches the format.

Important methods:

- AgentController.exportReport()
- ReportService.export()
- ReportService.writePdf()
- ReportService.writeCsv()
- ReportService.writeMarkdown()

Execution:

The user chooses the report type, the month, the format, and a destination path, then clicks Save. The GUI calls AgentController.exportReport(type, month, format, path), which calls ReportService.export(). The service collects the report data and checks the path. If the path is invalid, it returns an error and asks for a new one. If a file already exists there, it asks the user whether to overwrite it. Otherwise it calls the helper that matches the format: writePdf(), writeCsv(), or writeMarkdown(). The file location is returned through the controller and shown by the GUI.